实验 7 I/O 多路复用

实验目的

1、掌握基于 select()的 I/O 多路复用技术编写网络通信程序。

实验环境

联网并运行 Linux 操作系统的 PC 机

实验内容

本程序基于 select I/O 模型，设计并实现使用流式套接字、采用 C/S 交互模式的网络通信程序。要求如下：

1、服务器端使用链表方式来组织和管理与客户端已建立连接的连接套接字；

2、服务器端通过调用 select()函数进入等待状态，有客户端连接请求到达或者有客户发来数据时，从等待状态退出，然后对客户端的连接请求或者是发来的消息进行处理；

3、服务器端把接收到的客户端消息原样发回给客户端；

4、服务器端通过键盘接收到用户输入结束程序运行的命令后，会向所有客户端发送结束服务的消息，然后再关闭所有与客户端的连接，最后结束整个程序的运行；

5、客户端可以采用任意技术来实现。

实验指导

1、select()函数

I/O 多路复用通过调用 select()系统调用来实现。所以，有时候也把 I/O 多路复用方式称为 select I/O。I/O 多路复用使用 select()函数原型如下：

```
int select (
    int nfds,
    fd_set * readset,
    fd_set * writeset,
    fd_set * exceptset,
    const struct timeval * timeout
)
```

select 函数参数说明：

- int nfds：文件描述符的最大取值 maxfdp 再加 1。
- readset：指向套接字集合的指针。要求检查该集合中的套接字是否有数据可读。
- writeset：指向套接字集合的指针。要求检查该集合中的套接字是否可写。
- exceptset：指向套接字集合的指针。要求检查该集合中的套接字是否有异常数据。
- timeout：指向设置的超时时间的指针。

其中指针 timeout 指向一个 timeval 结构变量，该变量的值设定了 select()函数最长的等待时间。若该指针为 NULL，那么 select()函数会进入到无限等待状态，直到由 readset、writeset 或者 exceptset 所指向的套接字集合中至少有一个套接字上发生了期待的事件为止。若 timeout 指针指向的 timeval 结构体两个成员都为 0，那么 select()检测套接字描述符后就会立即返回，而不会进入到阻塞状态。

调用 select()函数时：readset、writeset、exceptset 这三个指针不能同时为空；而且，不为空的套接字集合中至少要有一个套接字描述符。select 函数对 readset、writeset、exceptset 三个指针所指向集合中的套接字进行检查，看是否有数据可读、可写或有异常（带外）数据，如果：至少一个套接字符合条件，就立即返回。符合条件的套接字仍在集合中，不符合条件的套接字则被删去。如果一个也没有，则等待。但最多等待 timeout 所指定的时间，便返回。

select 函数的返回值的含义为：

- -1：调用 select()时出现错误；
- 大于 0：readset 集合中有某些套接字可读，或者 writeset 集合中有某些套接字可写，或者 exceptset 集合中有某些套接字出现异常数据。其值为已准备好的套接字描述符的个数。
- 0：在 timeout 指定的时间内，所有套接字集合中没有一个套接字有数据可读、可写或者出现异常数据，select()函数超时返回。

I/O 多路复用使用数据结构 fd_set 来把套接字组织成为一个集合。每一个集合能够存储的套接字描述符数量的上限由常量 FD_SETSIZE 限制。系统中有如下定义：

```
#define __FD_SETSIZE  1024
#define FD_SETSIZE     __FD_SETSIZE
```

所以，在 Linux 下，每个 fd_set 可以保存 1024 个套接字描述符。虽然可以通过重新定义此常量而改变 fd_set 可以保存的套接字描述符最大数量。但是，修改常量后需要重新编译 Linux 内核才会生效，所以一般并不建议修改该常量的值。

为了方便编程，系统提供了四个宏来对套接字集合 fd_set 进行操作：

- FD_ZERO(fd_set *fdset)：初始化 fdset 为空集合。
- FD_SET(int fd, fd_set *fdset)：把套接字 fd 添加到套接字集合 fdset 中。
- FD_CLR(int fd, fd_set *fdset)：从套接字集合 fdset 中把套接字 fd 移除。
- FD_ISSET(int fd, fd_set *fdset)：检查 fd 是不是套接字集合 fdset 的成员，如果是返回 TRUE。

2、select I/O 编程框架

如果出现以下两种情况，就会从 select 函数中返回。select I/O 编程框架如图 3 所示。

（1）readset 指针所指向套接字集合的任意一个套接字上发生事件：接收缓存区中收到的数据大于或等于当前接收缓存数据的最低阈值、TCP 连接的对端关闭了连接、套接字发生了错误，或者当前套接字为监听套接字，并且已经与客户端成功建立起连接。

（2）writeset 指针所指向套接字集合的任意一个套接字上发生事件：当前套接字发送缓存空闲区大于或等于发送缓存的最小阈值、TCP 连接已关闭写操作、使用非阻塞模式的套接字已经建立了连接或者连接已失败、当前套接字已发生了错误。

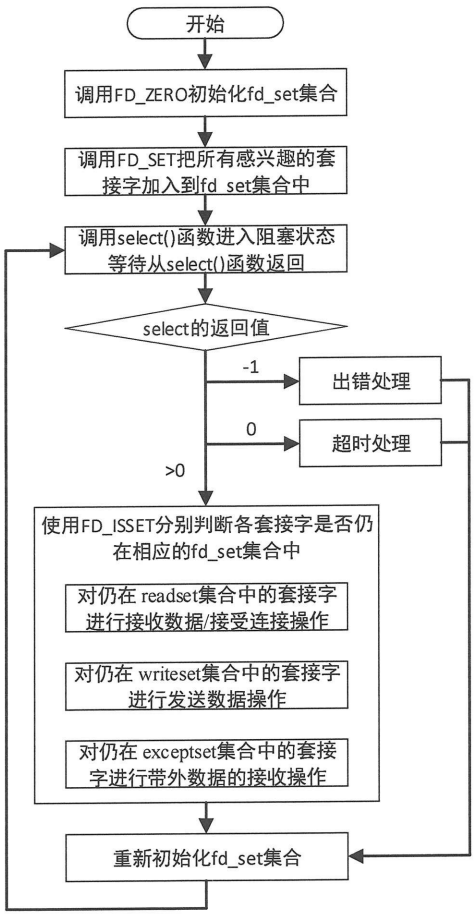


图 3-1 select I/O 编程框架

五、实验步骤

根据实验要求，设计出程序主函数工作流程如图 所示。

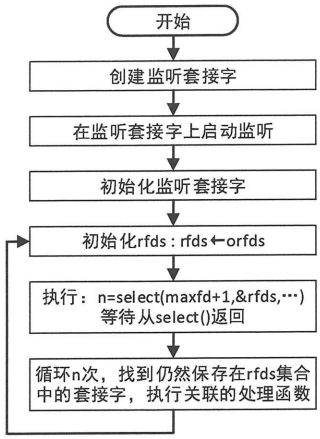


图 7-2 主函数流程图

根据如图 所示的主函数流程，设计出以下代码。

```
int main(int argc, char *argv[])
{
    int listen_fd;                          /*用于监听的套接字描述符*/
    struct sockaddr_in srvaddr;             /*监听套接字地址*/
    int port;
    int n;
    P_SKOPT p_so;
    socklen_t optlen;
    int sr = 1;

    if (argc != 2)
    {
        fprintf (stderr,"usage: %s port\a\n", argv[0]);
        exit (1);
    }

    if ((port = atoi(argv[1])) < 0)
    {
        fprintf (stderr, "Usage: %s port\a\n", argv[0]);
        exit (1);
    }

    /*创建Internet监听套接字*/
    if ((listen_fd = socket (PF_INET, SOCK_STREAM, 0)) == -1)
        bail("socket()");
    /*设置套接字选项*/
    optlen = sizeof(sr);
    int ret = setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &sr, optlen);
    if (ret)
```

```
        bail("setsockopt()");
    /*服务器监听地址准备*/
    memset (&srvaddr, 0, sizeof(srvaddr));
    srvaddr.sin_family = PF_INET;
    srvaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    srvaddr.sin_port = htons(port);
    /*绑定服务器到监听套接字*/
    if (bind (listen_fd, (struct sockaddr*) (&srvaddr), sizeof(srvaddr)) == -1)
        bail("bind()");
    /*开始监听*/
    if(listen (listen_fd, 5) == -1)
        bail("listen()");
    maxfd = listen_fd;
    /*初始化*/
    if (init (listen_fd))
        bail ("init");
    printf ("Server is waiting for acceptance of new client!\n");
    for (;;)
    {
        /*恢复rfds为最初设置的orfds，准备下次select的调用*/
        rfds = orfds;
        /*执行select*/
        n = select(maxfd + 1, &rfds, NULL, NULL, NULL);
        /*在返回的n个可读的文件描述符中进行迭代*/
        for( ; n > 0; n--)
        {
            /*在文件描述符选项链表中进行迭代*/
            list_for_each_entry(p_so, &fd_queue,list)
            {
                /*检测此套接字文件描述符是否被置位*/
                if(FD_ISSET( p_so->fd, &rfds))
                {
                    FD_CLR(p_so->fd, &rfds);
                    p_so->do_task(p_so);        /*执行对应的处理*/
                    break;
                }
            }
        }
    }
    close(listen_fd);
    return 0;
}
```

1、声明部分全局变量如下：

```
#define BUFLEN 256
#define PORT 20000
#define IPADDRLEN 16
int sndsock = -1;                    //发送消息使用的套接字
int rcvsock = -1;                    //接收消息使用的套接字
pthread_t rmsgthread = -1;           //接收消息的线程标识符
struct sockaddr_in destip;           //消息接收方地址
socklen_t addrlen;
struct ip_mreq grpip;                //加入的多播组地址
```

9、可能需要包含以下的头文件：

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <signal.h>
#include <pthread.h>
#include <fcntl.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
```